

Mask R-CNN 实例分割面试修改指南

基于你的 detection 代码改成 segmentation 任务后的面试复习版

这份指南总结了在面试中拿到 Data 数据集后，如何基于当前 Mask R-CNN instance segmentation 模板快速修改并跑通任务。当前模板默认使用 PennFudanPed 数据集、PNGImages 图像文件夹和 PedMasks mask 文件夹，并使用 maskrcnn_resnet50_fpn。

1. 先判断 segmentation 类型

类型	目标	常用模型	是否适合当前模板
Semantic Segmentation	给每个像素分类，例如 background / road / person	U-Net, FCN, DeepLabV3, SegFormer	不完全适合；更建议换 U-Net / DeepLabV3
Instance Segmentation	分割每一个独立目标实例，并输出 boxes / labels / masks / scores	Mask R-CNN	非常适合当前模板

你的当前代码属于 instance segmentation，因为 target 中包含 boxes、labels、masks、image_id、area 和 iscrowd，并且模型使用 Mask R-CNN。

2. 面试时最常改的配置区

如果面试给你的数据集文件夹叫 Data，通常需要先修改路径和类别。

```
# 将 PennFudan demo
DATA_ROOT = "data"
DATASET_NAME = "PennFudanPed"

# 将 Data
DATASET_DIR = "Data"
OUTPUT_DIR = "outputs_segmentation"
```

如果面试已经给了 Data 文件夹，不需要再下载数据。main() 中应把 download_and_extract_dataset() 替换成：

```
dataset_dir = "Data"
```

3. 类别名称和 NUM_CLASSES

Mask R-CNN 的类别 0 必须是 background，真实目标类别从 1 开始。

```
# 将 pedestrian
CLASS_NAMES = ["background", "pedestrian"]
NUM_CLASSES = len(CLASS_NAMES)

# 将 cat, dog
CLASS_NAMES = ["background", "cat", "dog"]
NUM_CLASSES = len(CLASS_NAMES)
```

不要写成 CLASS_NAMES = ["cat", "dog"]，因为缺少 background。

4. 数据集文件夹名称需要灵活修改

当前 Dataset 写死了 PennFudan 的文件夹名称：

```
self.imgs = sorted(os.listdir(os.path.join(root, "PNGImages")))
self.masks = sorted(os.listdir(os.path.join(root, "PedMasks")))

img_path = os.path.join(self.root, "PNGImages", self.imgs[idx])
mask_path = os.path.join(self.root, "PedMasks", self.masks[idx])
```

如果面试数据结构是 Data/images 和 Data/masks，需要改成：

```
self.imgs = sorted(os.listdir(os.path.join(root, "images")))
self.masks = sorted(os.listdir(os.path.join(root, "masks")))

img_path = os.path.join(self.root, "images", self.imgs[idx])
mask_path = os.path.join(self.root, "masks", self.masks[idx])
```

5. mask 读取方式怎么改

情况 A：instance id mask

如果 mask 中 0 是背景，1/2/3 是不同实例，则当前逻辑基本不用改：

```
mask = np.array(Image.open(mask_path))
obj_ids = np.unique(mask)
obj_ids = obj_ids[1:] # remove background
masks = mask == obj_ids[:, None, None] # [N, H, W]
```

情况 B：binary mask

如果 mask 中 0 是背景，255 是目标，且只有一个前景类别，可以保留 obj_ids 逻辑，并把 label 都设为 1。

```
labels = torch.ones((boxes.shape[0],), dtype=torch.int64)
```

情况 C：RGB color mask

如果 mask 是彩色图，需要先把颜色转换为类别 ID。

```
mask_rgb = np.array(Image.open(mask_path).convert("RGB"))
class_mask = np.zeros(mask_rgb.shape[:2], dtype=np.uint8)

# red = cat, blue = dog
class_mask[(mask_rgb[:, :, 0] == 255) & (mask_rgb[:, :, 1] == 0) & (mask_rgb[:, :, 2] == 0)] = 1
class_mask[(mask_rgb[:, :, 0] == 0) & (mask_rgb[:, :, 1] == 0) & (mask_rgb[:, :, 2] == 255)] = 2

mask = class_mask
```

6. labels 生成逻辑

当前模板默认所有目标都是 pedestrian，因此使用：

```
labels = torch.ones((boxes.shape[0],), dtype=torch.int64)
```

如果是多类别任务，例如 cat/dog，就需要根据 mask 或 annotation 生成 labels。最简单的 label_map 示例：

```
label_map = {"cat": 1, "dog": 2}
labels.append(label_map[class_name])
```

7. build_model 需要同时替换两个 head

Mask R-CNN 不仅需要 box predictor，还需要 mask predictor。面试时一定要说明这点。

```
def build_model(num_classes):
    model = maskrcnn_resnet50_fpn(weights="DEFAULT")

    # Replace box classification/regression head
    in_features_box = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features_box, num_classes)

    # Replace mask prediction head
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(
        in_features_mask,
        hidden_layer,
        num_classes
    )
    return model
```

8. 如果数据已经分成 train/val

当前模板用 TRAIN_RATIO 随机划分。如果 Data 已经有 train/val 文件夹，就不要随机 split。

```
train_dataset = CustomSegmentationDataset(
    root=os.path.join("Data", "train"),
    transforms=get_transform(train=True)
)

val_dataset = CustomSegmentationDataset(
    root=os.path.join("Data", "val"),
```

```

    transforms=get_transform(train=False)
)

```

9. CPU/GPU 面试推荐参数

环境	BATCH_SIZE	NUM_EPOCHS	说明
CPU 快速演示	1	1	只验证代码能跑通，Mask R-CNN 在 CPU 上较慢
GPU 面试环境	2	3	可以训练更充分，但仍以跑通流程为主
正式训练	2 或更大	更多 epoch	根据显存、数据量和时间调整

10. 通常不用改的部分

- collate_fn 必须保留，因为每张图目标数量不同。
- train_one_epoch 中不需要手写 loss；torchvision Mask R-CNN 会返回 loss_dict。
- run_inference 已经处理 boxes、labels、scores、masks，只需确保 CLASS_NAMES 正确。

```

loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())

```

11. 面试拿到 Data 后的检查步骤

```

print(os.listdir("Data"))

# Possible outputs:
# ['PNGImages', 'PedMasks']      -> current PennFudan-style Dataset works
# ['images', 'masks']           -> modify folder names in Dataset
# ['train', 'val']              -> build train_dataset and val_dataset separately

```

12. 面试中可以直接说的 plan

我会把这个任务作为 instance segmentation 处理，并使用 Mask R-CNN。首先检查数据结构，确认 annotation 是 instance mask、semantic mask 还是 COCO/polygon 格式。然后实现 PyTorch Dataset，使其返回 image 和 target 字典，其中包括 boxes、labels、masks、image_id、area 和 iscrowd。对于 Mask R-CNN，class 0 是 background，所以类别名必须包含 background。我会加载 pretrained Mask R-CNN，并根据目标类别数替换 box predictor 和 mask predictor。训练时 torchvision Mask R-CNN 会返回多个 loss，包括 classification、box regression、mask、objectness 和 RPN loss，我会把这些 loss 加起来反向传播，最后通过可视化 masks 和 boxes 评估结果。

13. 最终修改清单

- DATA_ROOT / dataset_dir：改成面试给出的 Data 路径。
- Dataset 中 image/mask 文件夹名称：PNGImages/PedMasks 或 images/masks。
- mask 读取方式：instance id mask、binary mask 或 RGB color mask。
- CLASS_NAMES：必须包含 background。
- NUM_CLASSES = len(CLASS_NAMES)。
- labels 生成逻辑：单类别全 1，多类别使用 label_map。
- 如果已有 train/val，不要随机 split。
- CPU 演示建议 BATCH_SIZE=1, NUM_EPOCHS=1。
- build_model 中 box predictor 和 mask predictor 都要替换。
- run_inference 中检查 score_threshold、mask_threshold 和 CLASS_NAMES。